ENGENHARIA DE SOFTWARE · EXTENSÃO I · ENGENHARIA DE COMPUTAÇÃO

Raiz Direta

Marketplace B2B que conecta produtores rurais a restaurantes, mercados e hortifrutis

Documentação Técnica

Ambiente de desenvolvimento e catálogo — Sprint 2

VERSÃO	DATA	RESPONSÁVEL	PÚBLICO
1.1	20/10/2026	Peres	Equipe do projeto

Documento de uso interno da equipe. Não contém senhas: cada integrante usa as credenciais do próprio arquivo .env.

Sumário

1 Sobre este documento 2
2 Visão geral do projeto 3
3 O que foi feito até agora 4
4 Arquitetura do ambiente 6
5 Tecnologias e versões 7
6 Estrutura do repositório 8
7 Como rodar o projeto (guia para a equipe) 9
8 Endereços de acesso 12
9 Modelo de dados do catálogo 13
10 API REST do catálogo 16
11 Dados de teste (fictícios) 18
12 Testes automatizados 19
13 Comandos do dia a dia 20
14 Segurança e boas práticas 22
15 Problemas comuns e soluções 23
16 Decisões técnicas registradas 24
17 Requisitos e backlog da Sprint 2 (proposta) 25
18 Situação atual e próximos passos 31
A Glossário 32

Histórico de versões

Versão	Data	Alterações
1.0	19/10/2026	Criação: ambiente, backend, catálogo, guia de instalação e decisões técnicas.
1.1	20/10/2026	Catálogo validado no computador do responsável (14 testes aprovados no PostgreSQL 18); repositório publicado no GitHub; nova seção 17 com a proposta de requisitos, histórias de usuário e backlog da Sprint 2.

1 Sobre este documento

Este documento registra o que já foi construído no Raiz Direta, explica como cada integrante pode rodar o projeto no próprio computador e reúne as informações importantes para o trabalho em equipe.

Para quem é

- **Integrantes que vão programar:** leiam as seções 4 a 8 e 13 (arquitetura, instalação, acessos e comandos).
- **Integrantes de documentação, design e extensão:** leiam as seções 2, 3, 9, 17 e 18 (visão geral, modelo de dados, requisitos e próximos passos).
- **Todos:** leiam a seção 14 (segurança). Ela explica o que **nunca** deve ser compartilhado.

Legenda de situação

Para separar com clareza o que foi feito do que ainda é plano, o documento usa estas marcações:

Marcação	Significado
CONCLUÍDO	Executado e confirmado no computador do responsável pelo projeto.
VERIFICADO	Implementado e aprovado em testes automatizados em um ambiente de verificação separado. Ainda falta repetir a execução no computador do responsável.
PENDENTE	Necessário e já definido, mas ainda não executado.
PLANEJADO	Proposta para etapas futuras. Ainda depende de decisão da equipe.
EM ANÁLISE	Alternativa ou proposta sendo avaliada. Ainda não foi aprovada.

Identificação

Projeto	Raiz Direta — marketplace B2B de produtos agrícolas
Disciplinas	Engenharia de Software e Extensão I — Engenharia de Computação
Versão do documento	1.1
Data	20 de outubro de 2026
Etapa	Preparação da Sprint 2 (apresentação prevista para o início de novembro de 2026)
Responsável técnico	Peres
Repositório	`github.com/Lucax-Peres/raiz-direta` (privado)
Elaboração	Documento e código produzidos com apoio de IA (Claude, da Anthropic), sob orientação e revisão do responsável técnico.

2 Visão geral do projeto

O Raiz Direta é um marketplace B2B que conecta pequenos e médios produtores rurais diretamente a compradores profissionais, como restaurantes, mercados e hortifrutis. A proposta é funcionar como uma espécie de CEASA digital, com foco inicial em Cuiabá e na região de Mato Grosso.

Objetivos centrais

- Facilitar a comercialização entre produtores e estabelecimentos.
- Ampliar o acesso dos produtores ao mercado e reduzir barreiras comerciais.
- Promover a inclusão digital no setor agrícola.
- Aplicar Engenharia de Software à solução de um problema real da comunidade.

Escopo da Sprint 2: catálogo

O professor orientou que cada Sprint mostre uma **parte funcional e crucial** do produto, em vez da visão geral. Para a Sprint 2, a parte escolhida foi o **catálogo**:

Dentro do escopo	Fora do escopo (etapas futuras)
• Cadastro de produtores, categorias e unidades • Cadastro de produtos com preço e disponibilidade • Consulta do catálogo com filtros, busca e ordenação • Painel administrativo (Django Admin) • Dados de teste fictícios e testes automatizados	• Tela de login e perfis de usuário • Carrinho e pedidos • Estoque por quantidade, frete e pagamento • Interface web para o usuário final (frontend)

> **Por que começar pelo catálogo?**
> Todas as outras funcionalidades dependem dele. Sem produtores e produtos bem modelados, não há carrinho nem pedido. Com um prazo curto e um único programador, entregar o catálogo completo e testado é mais seguro do que entregar várias partes incompletas.

3 O que foi feito até agora

Resumo, em ordem cronológica, das etapas realizadas em 19 e 20/10/2026.

#	Etapa	Resultado	Situação
1	Banco de dados em Docker	PostgreSQL 18 e Adminer configurados com Docker Compose. O banco fica acessível só pelo próprio computador.	CONCLUÍDO
2	Conflito de porta resolvido	O computador do responsável já tinha um PostgreSQL na porta 5432. O banco do projeto passou a usar a **5433**.	CONCLUÍDO
3	Teste de acesso	Contêiner do banco `healthy` e login no Adminer funcionando.	CONCLUÍDO
4	Repositório Git	Repositório local criado. Primeiro commit (`de9ccfe`) com a configuração do Docker. O arquivo de senhas (`.env`) ficou fora do Git, como deve ser.	CONCLUÍDO
5	Backend Django + DRF	Projeto Django 5.2 LTS com Django REST Framework, conectado ao banco. Migrations iniciais aplicadas, superusuário criado e Django Admin acessado.	CONCLUÍDO
6	Catálogo (app `catalogo`)	Modelo de dados, Django Admin, API somente leitura com filtros, comando de dados fictícios e 14 testes automatizados.	CONCLUÍDO
7	Catálogo no computador do responsável	Migration aplicada, dados fictícios criados, API acessada no navegador e **14 testes aprovados no PostgreSQL 18**. Commit `9a79c4e` .	CONCLUÍDO
8	Repositório no GitHub	Repositório privado `github.com/Lucax-Peres/raiz-direta` publicado e sincronizado. Antes do envio, o histórico foi conferido: nenhuma senha ou chave versionada.	CONCLUÍDO
9	Acesso da equipe	Convidar os integrantes como colaboradores no GitHub.	PENDENTE

Como o catálogo foi verificado

Os 14 testes automatizados foram executados em dois ambientes, e todos passaram nos dois: primeiro em um ambiente de verificação separado (PostgreSQL 16) e depois no computador do responsável, com o **PostgreSQL 18** do projeto rodando no Docker.

Uma correção encontrada pelos testes

Durante a verificação, a lista de produtores voltava na ordem de cadastro, e não em ordem alfabética. A causa é um comportamento documentado do Django: **em consultas com agregação (`Count`), o `ordering` definido no modelo é ignorado**. Sem uma ordem fixa, a paginação poderia repetir ou pular produtores entre as páginas. A correção foi declarar a ordem explicitamente na consulta, e um teste novo garante que o problema não volte.

4 Arquitetura do ambiente

Só o banco de dados roda dentro do Docker. O Django roda direto no computador, dentro de um ambiente virtual Python. Essa divisão mantém o ambiente simples e o banco fácil de recriar.

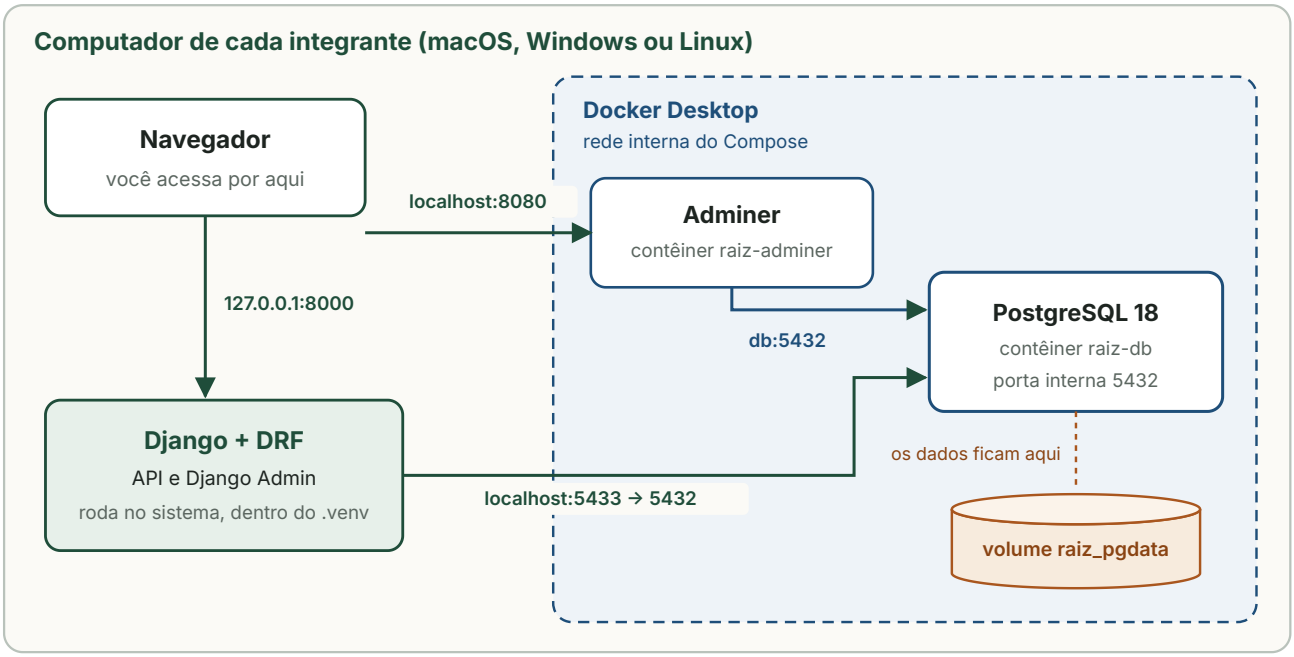


Figura 1. Componentes do ambiente de desenvolvimento e as portas usadas entre eles.

Como ler o diagrama

Caminho	Explicação
Navegador → Django	A API e o Django Admin ficam em `http://127.0.0.1:8000` enquanto o comando `runserver` estiver rodando.
Django → PostgreSQL	O Django está **fora** do Docker. Por isso ele usa a porta que o Docker expõe no computador (`localhost:5433`), que é redirecionada para a porta 5432 dentro do contêiner.
Navegador → Adminer	O Adminer é uma interface web para ver as tabelas, em `http://localhost:8080` .
Adminer → PostgreSQL	O Adminer está **dentro** do Docker. Lá, o banco é encontrado pelo nome do serviço (`db`) e pela porta padrão 5432.
Volume `raiz_pgdata`	Guarda os dados do banco. Ele sobrevive quando os contêineres são desligados e só é apagado com `docker compose down -v` .

A regra que mais causa confusão

De **fora** do Docker (Django, terminal, DBeaver), use `localhost` e a porta do `.env` , que hoje é a 5433. De **dentro** do Docker (Adminer), use `db` e a porta 5432. Trocar uma pela outra gera erros de conexão recusada.

5 Tecnologias e versões

Tecnologia	Versão	Papel no projeto e motivo da escolha
Docker Desktop	atual	Executa o banco em contêiner. Todos os integrantes usam o mesmo banco, na mesma versão, criado com um único comando.
PostgreSQL	18 (imagem `postgres:18`)	Banco relacional. É robusto, gratuito e garante integridade com chaves, CHECK e UNIQUE.
Adminer	imagem `adminer`	Interface web leve para inspecionar o banco durante o desenvolvimento.
Python	3.10 ou superior	Linguagem do backend. Versão mínima exigida pelo Django 5.2.
Django	5.2.x LTS	Framework web. A versão LTS recebe correções de segurança até abril de 2028, o que cobre todo o projeto acadêmico.
Django REST Framework	3.18.x	Cria a API REST com paginação, busca, ordenação e uma interface navegável para testar no navegador.
psycopg	3.x	Driver que conecta o Python ao PostgreSQL.
python-dotenv	1.x	Lê as configurações sensíveis do arquivo `.env`.
Git	—	Controle de versão, com mensagens no padrão Conventional Commits.
VS Code	—	Editor recomendado, com as extensões Python e Container Tools (sugeridas automaticamente ao abrir o projeto).

> **Por que as versões são fixadas por faixa?**
> O `requirements.txt` usa faixas como `Django>=5.2,<5.3`. Assim, todos recebem correções de segurança da 5.2, mas ninguém instala por acidente uma versão maior, que poderia ter mudanças incompatíveis. O mesmo vale para a imagem `postgres:18`.

6 Estrutura do repositório

```
raiz-direta/
├── docker-compose.yml          # PostgreSQL 18 + Adminer
├── .env                        # senhas e chaves — NUNCA vai para o Git
├── .env.example                # modelo do .env, sem valores reais
├── .gitignore                  # o que o Git deve ignorar
├── README.md                   # instruções rápidas
├── .vscode/
│   ├── extensions.json         # extensões recomendadas
│   └── settings.json           # aponta o VS Code para o Python do .venv
├── db/init/                    # scripts SQL iniciais (vazio por enquanto)
└── backend/
    ├── requirements.txt        # dependências Python
    ├── manage.py               # comandos do Django
    ├── .venv/                  # ambiente virtual — NÃO vai para o Git
    ├── config/                 # configuração geral do projeto
    │   ├── settings.py         # lê tudo do .env; seguro por padrão
    │   ├── urls.py             # rotas: /admin/, /api/health/, /api/...
    │   └── views.py            # endpoint de saúde (testa o banco)
    └── catalogo/               # app do catálogo (Sprint 2)
        ├── models.py           # tabelas e regras de integridade
        ├── admin.py            # telas do Django Admin
        ├── serializers.py      # formato JSON da API
        ├── views.py            # endpoints, filtros e busca
        ├── urls.py             # rotas do catálogo
        ├── tests.py            # 14 testes automatizados
        ├── migrations/         # histórico versionado do banco
        └── management/commands/
            └── popular_dados.py  # cria os dados fictícios
```

Organização em camadas

O backend segue a separação de responsabilidades do Django:

Arquivo	Camada	Responsabilidade
`models.py`	Dados	Define as tabelas e as regras que o próprio banco garante.
`serializers.py`	Contrato	Define quais campos saem em JSON e em que formato.
`views.py`	Aplicação	Recebe a requisição, aplica filtros e devolve a resposta.
`urls.py`	Roteamento	Liga cada endereço à sua view.
`admin.py`	Administração	Monta as telas de cadastro do Django Admin.

7 Como rodar o projeto (guia para a equipe)

Siga os passos na ordem. Na primeira vez, o processo leva de 20 a 40 minutos, principalmente pela instalação dos programas. Depois disso, basta usar os comandos da seção 13.

Pré-requisitos por sistema

Programa	macOS	Windows	Linux
Docker	Docker Desktop (docker.com)	Docker Desktop com WSL 2. Antes, rode `wsl --install` no PowerShell como administrador e reinicie.	Docker Engine + plugin Compose
Python 3.10+	`brew install python@3.13` . O Python que vem com o macOS costuma ser 3.9, que não serve.	Instalador do python.org. Marque **"Add python.exe to PATH"**.	Pacote da distribuição (`python3` , `python3-venv`)
Git	Já vem com as ferramentas do Xcode	git-scm.com	Pacote da distribuição
VS Code	code.visualstudio.com. Ao abrir o projeto, aceite as extensões recomendadas.		

> **Antes de começar**
>
> O Docker Desktop precisa estar **aberto e rodando**. O ícone da baleia deve aparecer parado na barra de menu ou na bandeja do sistema.

1 Obter o código

O repositório é **privado**. Primeiro, aceite o convite de colaborador que chega por e-mail (o responsável envia pelo GitHub). Depois:

```
git clone https://github.com/Lucax-Peres/raiz-direta.git
cd raiz-direta
```

Se o Git pedir senha, o GitHub não aceita a senha da conta no terminal. O jeito mais simples é instalar o GitHub CLI (cli.github.com) e rodar `gh auth login` , escolhendo HTTPS e o login pelo navegador.

2 Criar o seu arquivo `.env`

```
# macOS / Linux
cp .env.example .env
# Windows (PowerShell)
Copy-Item .env.example .env
```

Abra o `.env` no VS Code e preencha:

- `POSTGRES_PASSWORD`: uma senha **sua**, só com letras e números. Repita a mesma senha dentro da `DATABASE_URL`.
- `DJANGO_SECRET_KEY`: gere uma chave com o comando abaixo e cole o resultado.
- `POSTGRES_PORT`: deixe `5432`. Se essa porta estiver ocupada no seu computador, troque para `5433` (veja a seção 15).

```
# macOS / Linux
python3 -c "import secrets; print(secrets.token_urlsafe(50))"
# Windows (PowerShell)
python -c "import secrets; print(secrets.token_urlsafe(50))"
```

3 Subir o banco de dados

```
docker compose up -d
docker compose ps
```

Espere o serviço `raiz-db` aparecer como **(healthy)**. Na primeira vez, o Docker baixa as imagens, o que leva alguns minutos.

4 Criar o ambiente virtual Python e instalar as dependências

```
# macOS / Linux
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt

# Windows (PowerShell)
cd backend
py -m venv .venv
.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

Com o ambiente ativo, o terminal mostra `(.venv)` no início da linha. **Sempre ative o ambiente** antes de usar comandos do Django.

5 Criar as tabelas no banco

```
python manage.py migrate
```

Aplica as migrations: cria as tabelas internas do Django e as do catálogo. Cada linha deve terminar com `OK`.

6 Popular os dados de teste (fictícios)

```
python manage.py popular_dados
```

Resultado esperado: `Catálogo pronto: 5 categorias, 5 unidades, 5 produtores, 20 produtos (dados FICTÍCIOS).`

7 Criar o seu usuário administrador

```
python manage.py createsuperuser
```

Esse usuário existe só no **seu** banco local. Cada integrante cria o próprio.

8 Rodar os testes automatizados

```
python manage.py test catalogo
```

Resultado esperado: `Ran 14 tests` seguido de `OK`.

9 Iniciar o servidor

```
python manage.py runserver
```

Acesse os endereços da seção 8. Para parar o servidor, aperte **Control + C** no terminal. No Mac, é a tecla *Control*, não *Command*.

> **Dica: use dois terminais**
>
> Enquanto o `runserver` estiver rodando, aquele terminal fica ocupado. Abra um segundo terminal no VS Code (botão **+** no painel do terminal) para Git e outros comandos, lembrando de ativar o `.venv` nele também.

8 Endereços de acesso

Estes endereços funcionam apenas no computador onde o projeto está rodando, porque todas as portas estão presas a `127.0.0.1`.

Serviço	Endereço	Como entrar
Verificação da API	`http://127.0.0.1:8000/api/health/`	Acesso livre. Deve mostrar `"status": "ok"` e `"banco": "conectado"`.
API do catálogo	`http://127.0.0.1:8000/api/`	Leitura livre. A página lista todos os endpoints.
Django Admin	`http://127.0.0.1:8000/admin/`	Usuário e senha criados no passo 7.
Adminer	`http://localhost:8080`	Sistema: **PostgreSQL**. Servidor: `db`. Usuário, senha e base: os valores do seu `.env`.
PostgreSQL (ferramentas externas)	`localhost`, porta do `.env`	Para DBeaver, TablePlus etc. No computador do responsável, a porta é a **5433**.

Credenciais: onde ficam

Credencial	Onde está
Senha do banco	Variável `POSTGRES_PASSWORD` no seu `.env`
Chave secreta do Django	Variável `DJANGO_SECRET_KEY` no seu `.env`
Login do Django Admin	Criado por você no passo 7, guardado no seu banco local

> **Senhas não aparecem neste documento, de propósito**
>
> Cada integrante cria as próprias senhas no próprio `.env`. Não envie o `.env` por WhatsApp, e-mail ou chat. Veja a seção 14.

Dica do macOS: copiar a senha do banco sem erro

```
grep '^POSTGRES_PASSWORD=' .env | cut -d= -f2 | tr -d '\n' | pbcopy
```

O comando copia a senha para a área de transferência sem mostrá-la na tela. Depois, é só colar no Adminer com **Cmd + V**.

9 Modelo de dados do catálogo

Quatro tabelas organizam o catálogo. Um produtor oferta produtos, cada produto pertence a uma categoria e é vendido em uma unidade de medida.

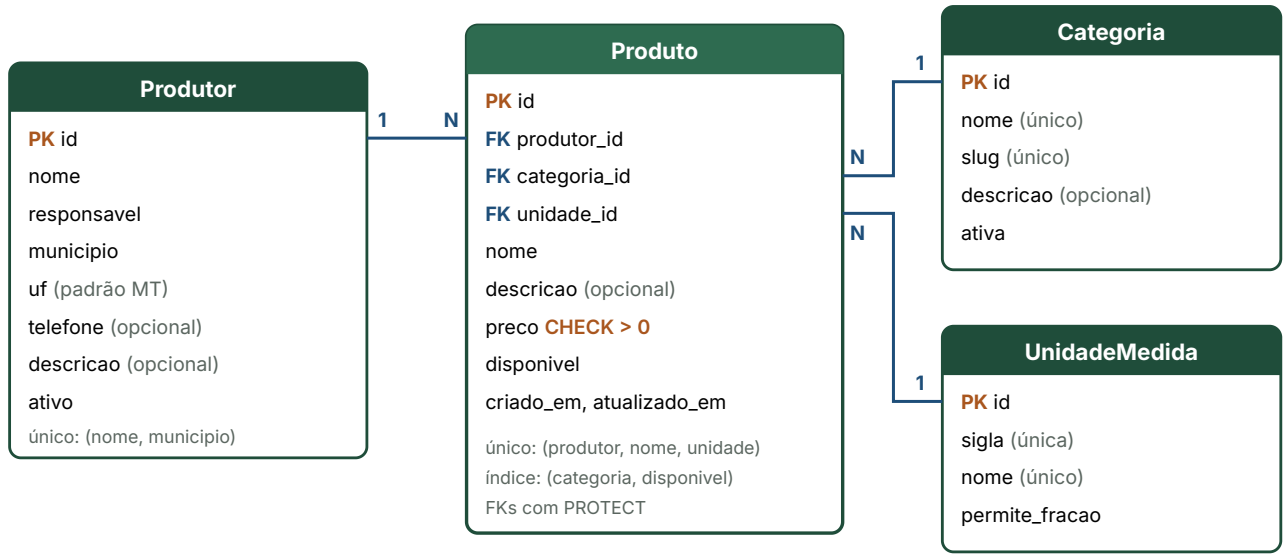


Figura 2. Diagrama entidade-relacionamento do catálogo (Sprint 2).

Relacionamentos

Relação	Cardinalidade	Leitura
Produtor → Produto	1 : N	Um produtor oferta vários produtos. Cada produto pertence a um único produtor.
Categoria → Produto	1 : N	Uma categoria agrupa vários produtos.
UnidadeMedida → Produto	1 : N	Uma unidade (kg, maço, dúzia...) é usada por vários produtos.

Dicionário de dados

Produto

Campo	Tipo	Regra / descrição
`id`	BIGINT	Chave primária, gerada automaticamente.
`produtor_id`	BIGINT (FK)	Produtor que oferta o item. Obrigatório.
`categoria_id`	BIGINT (FK)	Categoria do item. Obrigatório.
`unidade_id`	BIGINT (FK)	Unidade de venda. Obrigatório.

Campo	Tipo	Regra / descrição
`nome`	VARCHAR(120)	Nome do produto. Obrigatório.
`descricao`	TEXT	Opcional.
`preco`	NUMERIC(10,2)	Preço em reais. **Precisa ser maior que zero**, regra garantida pelo banco.
`disponivel`	BOOLEAN	Padrão: verdadeiro. Desmarque quando o item estiver fora de época ou esgotado.
`criado_em` / `atualizado_em`	TIMESTAMPTZ	Preenchidos automaticamente, gravados em UTC.

Produtor

Campo	Tipo	Regra
`nome`	VARCHAR(120)	Obrigatório
`responsavel`	VARCHAR(120)	Obrigatório
`municipio`	VARCHAR(80)	Obrigatório
`uf`	VARCHAR(2)	Padrão "MT"
`telefone`	VARCHAR(20)	Opcional
`descricao`	TEXT	Opcional
`ativo`	BOOLEAN	Padrão: sim

Único: a combinação (nome, município).

Categoria

Campo	Tipo	Regra
`nome`	VARCHAR(60)	Único
`slug`	VARCHAR(60)	Único; usado nos filtros
`descricao`	TEXT	Opcional
`ativa`	BOOLEAN	Padrão: sim

UnidadeMedida

Campo	Tipo	Regra
`sigla`	VARCHAR(10)	Única (kg, maço...)
`nome`	VARCHAR(40)	Único
`permite_fracao`	BOOLEAN	Ex.: 2,5 kg

Regras de integridade e por que existem

Regra	Justificativa
CHECK preço > 0	Impede produto com preço zero ou negativo, **mesmo** que o dado chegue por fora do formulário (script, outro sistema, erro de código).
UNIQUE (produtor, nome, unidade)	O mesmo produtor não anuncia o mesmo item duas vezes na mesma unidade. Vender "Alface" por maço e por kg continua permitido.
UNIQUE (nome, município) do produtor	Evita cadastrar o mesmo produtor em duplicidade.
PROTECT nas chaves estrangeiras	Não deixa apagar um produtor, uma categoria ou uma unidade que ainda tenha produtos. Isso protege o histórico; para retirar do catálogo, use os campos `ativo` / `ativa`.
Índice (categoria, disponivel)	Acelera a consulta mais comum do catálogo: os produtos disponíveis de uma categoria.
Sem CPF, CNPJ ou endereço	Princípio de minimização de dados pessoais (LGPD). Só se guarda o necessário para o catálogo.

10 API REST do catálogo

A API é **somente leitura** nesta Sprint. O cadastro e a edição são feitos pelo Django Admin, porque ainda não há tela de login. Todas as listas de produtos e produtores são paginadas, com 20 itens por página.

Endpoints

Método	Endereço	Retorna
GET	`/api/health/`	Se a API está no ar e o banco responde
GET	`/api/`	Lista navegável de todos os endpoints
GET	`/api/categorias/`	Categorias ativas (sem paginação)
GET	`/api/unidades/`	Unidades de medida (sem paginação)
GET	`/api/produtores/`	Produtores ativos, em ordem alfabética, com o total de produtos disponíveis
GET	`/api/produtos/`	Produtos de produtores ativos, com filtros (abaixo)
GET	`/api/<recurso>/<id>/`	Um item específico (ex.: `/api/produtos/12/`)

Filtros de `/api/produtos/`

Parâmetro	Efeito	Exemplo
`categoria=<slug>`	Só uma categoria	`?categoria=frutas`
`produtor=<id>`	Só um produtor	`?produtor=3`
`municipio=<nome>`	Produtores de um município (sem diferenciar maiúsculas)	`?municipio=Cuiabá`
`disponivel=true|false`	Disponíveis ou indisponíveis	`?disponivel=true`
`search=<texto>`	Busca no nome, na descrição e no nome do produtor	`?search=ovos`
`ordering=<campo>`	Ordena por `preco`, `nome` ou `criado_em`. O sinal `-` inverte a ordem.	`?ordering=-preco`
`page=<n>`	Página da lista	`?page=2`

Os filtros podem ser combinados com `&`. Por exemplo, frutas disponíveis da mais cara para a mais barata:

```
GET /api/produtos/?categoria=frutas&disponivel=true&ordering=-preco
```

Exemplo de resposta (real, dos dados fictícios)


```
{
  "count": 3,
  "next": null,
  "previous": null,
  "results": [
    {
      "id": 12,
      "nome": "Maracujá",
      "descricao": "",
      "preco": "8.50",
      "disponivel": true,
      "produtor": { "id": 3, "nome": "Sítio Serra Azul (fictício)",
                    "municipio": "Chapada dos Guimarães", "uf": "MT" },
      "categoria": { "id": 2, "nome": "Frutas", "slug": "frutas",
                     "descricao": "Frutas da estação e regionais." },
      "unidade": { "id": 1, "sigla": "kg", "nome": "Quilograma",
                   "permite_fracao": true }
    },
    ... (mais 2 produtos: Manga e Banana-nanica)
  ]
}
```


Permissões

Ação	Quem pode
Consultar (GET)	Qualquer pessoa
Criar, editar ou excluir pela API	Ninguém nesta Sprint. Esses endpoints ainda não existem, e um visitante que tentar recebe `403 Forbidden`.
Criar, editar ou excluir pelo Admin	Usuários com acesso ao Django Admin

O preço sai como texto (`"8.50"`) de propósito. Valores decimais exatos evitam erros de arredondamento com dinheiro, que aconteceriam com números de ponto flutuante.

11 Dados de teste (fictícios)

> **Atenção: todos os dados são inventados**
>
> Os produtores levam a marcação **"(fictício)"** no nome, os responsáveis têm nomes de exemplo e os preços são ilustrativos. Eles **não** representam produtores, parcerias, entrevistas ou preços reais e não devem ser apresentados como evidência de atividade de extensão.

O que o comando `popular_dados` cria

Item	Qtd.	Conteúdo
Categorias	5	Hortaliças, Frutas, Raízes e Tubérculos, Ovos, Temperos e Ervas
Unidades	5	kg (aceita fração), maço, dúzia, unidade, caixa
Produtores	5	Um em cada município: Cuiabá, Várzea Grande, Chapada dos Guimarães, Santo Antônio de Leverger e Nossa Senhora do Livramento
Produtos	20	18 disponíveis e 2 indisponíveis (mamão e pequi, para testar o filtro de disponibilidade)

Comportamento do comando

- **Pode ser repetido sem duplicar dados.** Se um item já existe, ele é atualizado em vez de criado de novo. Um teste automatizado garante isso.
- **Tudo ou nada.** O comando roda dentro de uma transação. Se algo falhar no meio, nada fica gravado pela metade.
- `--limpar` apaga todo o catálogo antes de recriar, útil para voltar ao estado inicial.

Roteiro para deixar tudo pronto antes de uma apresentação

```
docker compose down -v                # apaga o banco inteiro (inclusive o superusuário)
docker compose up -d                  # recria o banco vazio
cd backend && source .venv/bin/activate
python manage.py migrate              # recria as tabelas
python manage.py popular_dados        # recria os dados fictícios
python manage.py createsuperuser      # recria o seu login do Admin
```

Se quiser apenas restaurar o catálogo, sem apagar o login, use só `python manage.py popular_dados --limpar`.

12 Testes automatizados

Os testes conferem automaticamente se as regras continuam valendo depois de cada mudança no código. O Django cria um banco de teste temporário, roda os testes e apaga esse banco no final, então os seus dados não são afetados.

```
python manage.py test catalogo -v 2     # -v 2 mostra o nome de cada teste
```

Grupo	O que é verificado
Regras do banco (4)	• Preço zero é rejeitado pelo banco • Mesmo produto, mesmo produtor e mesma unidade não se repetem • Mesmo produto em outra unidade é permitido • Produtor com produtos não pode ser apagado
API (9)	• Endpoint de saúde responde e confirma o banco • A lista de produtos é pública e paginada • Filtros por categoria e por disponibilidade • Busca por texto e ordenação por preço • Produtos de produtor inativo somem do catálogo • Produtores em ordem alfabética, com a contagem correta • Visitante não consegue criar produto
Dados fictícios (1)	O comando pode rodar duas vezes sem duplicar, cria 20 produtos e todos os produtores estão marcados como fictícios.

Resultado da verificação

`Ran 14 tests ... OK`. Aprovados no ambiente de verificação (PostgreSQL 16) e, em 20/10/2026, no computador do responsável (PostgreSQL 18 no Docker).

Boas práticas para a equipe

- Rode os testes **antes de cada commit** que mexa no backend.
- Cada regra de negócio nova precisa de pelo menos um teste que comprove o comportamento esperado.
- Se um teste falhar, não o apague: descubra se o erro está no código ou no próprio teste.

13 Comandos do dia a dia

Docker (na pasta `raiz-direta`)

Comando	Para que serve
`docker compose up -d`	Liga o banco e o Adminer
`docker compose ps`	Mostra o status (procure *healthy*)
`docker compose logs -f db`	Acompanha os logs do banco
`docker compose down`	Desliga tudo e **mantém** os dados
`docker compose down -v`	Desliga tudo e **apaga** o banco (reset total)

Django (na pasta `backend`, com o `.venv` ativo)

Comando	Para que serve
`source .venv/bin/activate`	Ativa o ambiente virtual (Windows: `.venv\Scripts\Activate.ps1`)
`python manage.py runserver`	Inicia o servidor. Para parar: **Control + C**
`python manage.py makemigrations`	Gera uma migration depois de alterar um `models.py`
`python manage.py migrate`	Aplica as migrations no banco
`python manage.py popular_dados`	Cria ou atualiza os dados fictícios
`python manage.py test catalogo`	Roda os testes do catálogo
`python manage.py createsuperuser`	Cria um login para o Django Admin
`python manage.py check`	Procura erros de configuração

Git (na pasta `raiz-direta`)

Comando	Para que serve
`git status`	Mostra o que mudou. Confira que o `.env` e o `.venv` **não** aparecem.
`git add .`	Prepara as mudanças para o commit
`git commit -m "tipo: descrição"`	Registra as mudanças
`git log --oneline`	Mostra o histórico de commits

Padrão das mensagens de commit (Conventional Commits)

Prefixo	Quando usar	Exemplo
`feat:`	Funcionalidade nova	`feat: adiciona filtro por município`
`fix:`	Correção de erro	`fix: ordena produtores alfabeticamente`
`docs:`	Documentação	`docs: atualiza guia de instalação`
`test:`	Testes	`test: cobre busca por produtor`
`chore:`	Infraestrutura e configuração	`chore: atualiza dependências`

14 Segurança e boas práticas

Regras para todos

Regra	Por quê
Nunca envie o `.env` pelo Git, WhatsApp, e-mail ou chat	Ele guarda a senha do banco e a `SECRET_KEY`. Com a chave secreta, alguém consegue forjar sessões de login.
Cada integrante cria as próprias senhas	Se uma senha vazar, só aquele ambiente local é afetado.
Confira o `git status` antes de todo commit	É a última barreira antes de um segredo ir para o GitHub. Depois de publicado, ele precisa ser considerado comprometido.
Não use senhas reais pessoais no projeto	As credenciais locais são só de desenvolvimento.
Dados fictícios ficam identificados	Integridade acadêmica: dado inventado nunca pode parecer evidência real.

Proteções já implementadas

- **Portas presas a `127.0.0.1`:** o banco e o Adminer não ficam visíveis para outras pessoas na mesma rede Wi-Fi, como a da faculdade.
- **Configurações sensíveis vêm do `.env`:** nada de senha escrita no código. Se faltar uma variável, o sistema para com uma mensagem clara.
- **`DEBUG` desligado por padrão:** se o `.env` faltar em produção, o sistema não exibe páginas de erro com detalhes internos.
- **Escrita bloqueada para visitantes:** a API só permite leitura. Alterações exigem login no Admin.
- **Regras no próprio banco:** CHECK, UNIQUE e PROTECT valem mesmo se o código tiver um erro.
- **Mínimo de dados pessoais:** sem CPF, CNPJ ou endereço no catálogo (LGPD).

> **Antes de publicar na internet (ainda não é o caso)**
>
> O comando `python manage.py check --deploy` aponta hoje 5 avisos esperados para um ambiente local, todos ligados a HTTPS e ao `DEBUG`. Eles precisam ser resolvidos antes de qualquer publicação: `DEBUG=False`, HTTPS obrigatório e cookies seguros.

15 Problemas comuns e soluções

Problemas reais encontrados durante a configuração, além de outros previsíveis para quem está começando.

Mensagem ou sintoma	Causa	Solução
`ports are not available` / `address already in use` na 5432	Já existe outro PostgreSQL instalado no computador	No `.env`, troque `POSTGRES_PORT` para `5433` e ajuste a porta na `DATABASE_URL`. Para ver quem ocupa a porta: `lsof -nP -iTCP:5432 -sTCP:LISTEN`
`password authentication failed for user "raiz"` no Adminer	Senha digitada ou colada com erro	Copie a senha do `.env` com cuidado (no Mac, use o comando `pbcopy` da seção 8)
Mudou a senha no `.env` e o login falha	A senha só é aplicada quando o volume é criado	`docker compose down -v` e depois `docker compose up -d`. **Isso apaga os dados.**
Terminal "travado" depois do `runserver`	O servidor ocupa o terminal enquanto roda	**Control + C**, ou use um segundo terminal
`Cannot connect to the Docker daemon`	O Docker Desktop está fechado	Abra o Docker Desktop e espere iniciar
`ModuleNotFoundError: No module named 'django'`	O `.venv` não está ativo	`source .venv/bin/activate` (Windows: `.venv\Scripts\Activate.ps1`)
`python3 --version` mostra 3.9 no Mac	Python antigo do sistema	`brew install python@3.13` e recrie o `.venv`
`ImproperlyConfigured: Variável de ambiente ausente no .env: ...`	Falta uma variável no `.env`	Compare o seu `.env` com o `.env.example` e preencha o que falta
`connection refused` ao rodar o Django	Banco desligado ou porta diferente	`docker compose ps` e confira se `POSTGRES_PORT` é a porta mostrada
Windows: "a execução de scripts foi desabilitada"	Política padrão do PowerShell	`Set-ExecutionPolicy -Scope CurrentUser RemoteSigned` e tente ativar o `.venv` de novo
`popular_dados` diz que a tabela não existe	Migrations não aplicadas	`python manage.py migrate` antes do `popular_dados`

16 Decisões técnicas registradas

Registro resumido das decisões de arquitetura (ADR). Cada uma traz o contexto, a decisão e as consequências, para que a equipe entenda o motivo e possa revisá-la no futuro.

Decisão	Contexto	Consequências
ADR-001 Docker só para o banco	Desenvolvedor único, prazo curto, necessidade de resetar dados para as demonstrações.	(+) Banco igual para todos e recriado com um comando. (−) Exige Docker Desktop. A aplicação roda fora do Docker para manter o desenvolvimento simples.
ADR-002 Django 5.2 LTS + DRF	Escolha do responsável por Django REST. É preciso estabilidade durante todo o semestre.	(+) Suporte de segurança até 04/2028, Django Admin pronto, muitos tutoriais. (−) Frontend e backend em linguagens diferentes (TypeScript e Python).
ADR-003 Porta configurável (5433)	Conflito com um PostgreSQL já instalado no computador do responsável.	(+) Os dois bancos convivem. (−) Cada integrante confere a própria porta no `.env`.
ADR-004 API somente leitura e cadastro pelo Admin	A Sprint 2 não terá tela de login.	(+) Menos superfície de ataque e menos código para testar. (−) Endpoints de escrita ficam para quando houver autenticação.
ADR-005 Regras no banco e `PROTECT`	Marketplace precisa de histórico consistente.	(+) Dados válidos mesmo com erro no código. (−) Apagar registros exige primeiro desativá-los ou remover os produtos.

Todas as decisões estão com situação **Aceita** e podem ser revistas pela equipe com uma nova justificativa.

17 Requisitos e backlog da Sprint 2 (proposta)

Esta seção liga o que foi construído aos artefatos de Engenharia de Software e de Scrum: Sprint Goal, histórias de usuário, requisitos, rastreabilidade com os testes, Sprint Backlog e Definition of Done.

> **Situação: proposta para a equipe aprovar**
>
> No Scrum, o Sprint Goal é definido pela Scrum Team na Sprint Planning, e a Definition of Done é um compromisso de toda a equipe. O conteúdo abaixo é uma **proposta inicial**, escrita a partir do que já foi implementado. Ele precisa ser revisado e aprovado pela equipe e, quando possível, validado com produtores e compradores reais.

Sprint Goal (proposto)

> **"Compradores conseguem consultar o catálogo de produtos de produtores locais, com filtros, busca e ordenação, a partir de dados mantidos pelo painel administrativo."**

O objetivo descreve **o valor entregue**, e não a lista de tarefas. Assim, ele continua válido mesmo que algum item do backlog mude durante a Sprint.

Histórias de usuário

Formato: **Como** [perfil], **quero** [funcionalidade], **para** [benefício].

ID	História	Requisito
HU01	**Como** comprador, **quero** ver a lista de produtos ofertados, **para** conhecer a oferta dos produtores locais.	RF001
HU02	**Como** comprador, **quero** filtrar os produtos por categoria, **para** encontrar rapidamente o tipo de item de que preciso.	RF002
HU03	**Como** comprador, **quero** buscar produtos pelo nome, **para** localizar um item específico.	RF003
HU04	**Como** comprador, **quero** ordenar os produtos por preço, **para** comparar ofertas.	RF004
HU05	**Como** comprador, **quero** ver apenas produtos disponíveis, **para** não escolher itens fora de época ou esgotados.	RF005
HU06	**Como** comprador, **quero** filtrar por produtor ou município, **para** priorizar fornecedores próximos ou de minha confiança.	RF006
HU07	**Como** comprador, **quero** ver a lista de produtores com a quantidade de produtos disponíveis, **para** conhecer os fornecedores da região.	RF007
HU08	**Como** administrador da plataforma, **quero** cadastrar e editar produtores, produtos, categorias e unidades em um painel, **para** manter o catálogo atualizado.	RF008

Itens técnicos, como o ambiente Docker, os dados fictícios e os testes, não são escritos como histórias de usuário, porque não entregam valor direto a um usuário final. Eles aparecem no Sprint Backlog como tarefas.

Critérios de aceitação (exemplos em Dado / Quando / Então)

História	Critério
HU02	**Dado** um catálogo com frutas e hortaliças, **quando** o comprador filtra pela categoria "frutas", **então** apenas frutas são exibidas.
HU04	**Dado** produtos de R$ 3,50 e R$ 6,00, **quando** o comprador ordena por preço decrescente, **então** o de R$ 6,00 aparece primeiro.
HU05	**Dado** um produto marcado como indisponível, **quando** o comprador filtra por disponíveis, **então** esse produto não aparece.
HU07	**Dado** um produtor com um produto disponível e um indisponível, **quando** a lista de produtores é exibida, **então** ele aparece em ordem alfabética com total igual a 1.
HU08	**Dado** um produtor com produtos cadastrados, **quando** o administrador tenta excluí-lo, **então** a exclusão é bloqueada para preservar o histórico.

Requisitos funcionais (RF)

ID	Descrição	Prioridade
RF001	O sistema deve listar os produtos de produtores ativos, com paginação de 20 itens por página.	Essencial
RF002	O sistema deve permitir filtrar produtos por categoria.	Essencial
RF003	O sistema deve permitir buscar produtos por texto no nome, na descrição e no nome do produtor.	Essencial
RF004	O sistema deve permitir ordenar produtos por preço, nome ou data de cadastro.	Importante
RF005	O sistema deve permitir filtrar produtos por disponibilidade.	Essencial
RF006	O sistema deve permitir filtrar produtos por produtor e por município.	Importante
RF007	O sistema deve listar os produtores ativos em ordem alfabética, com o total de produtos disponíveis de cada um.	Importante
RF008	O sistema deve oferecer um painel administrativo para manter produtores, produtos, categorias e unidades.	Essencial
RF009	O sistema deve informar se a API e o banco de dados estão funcionando.	Desejável

Regras de negócio (RN)

ID	Regra
RN001	O preço de um produto deve ser maior que zero.
RN002	Um produtor não pode anunciar o mesmo produto duas vezes na mesma unidade. O mesmo produto em unidades diferentes é permitido.
RN003	Produtor, categoria ou unidade com produtos vinculados não podem ser excluídos. Para retirá-los do catálogo, devem ser desativados.
RN004	Produtos de produtores inativos não aparecem no catálogo.
RN005	Não pode haver dois produtores com o mesmo nome no mesmo município.
RN006	Dados de demonstração devem ser identificados como fictícios.

Requisitos não funcionais (RNF)

ID	Atributo (ISO/IEC 25010)	Requisito
RNF001	Segurança	Visitantes só podem consultar dados. Qualquer alteração exige usuário autenticado.
RNF002	Segurança	Senhas e chaves não podem estar no código nem no repositório Git; devem vir do arquivo `.env`.
RNF003	Portabilidade	O ambiente deve poder ser recriado a partir de um clone do repositório, com Docker e os passos documentados.
RNF004	Manutenibilidade	As regras do catálogo devem ter testes automatizados executáveis com um único comando.
RNF005	Eficiência de desempenho	As listagens devem ser paginadas e evitar consultas repetidas ao banco por item (problema N+1).
RNF006	Segurança (privacidade)	O catálogo deve armazenar o mínimo de dados pessoais, sem CPF, CNPJ ou endereço (LGPD).

Matriz de rastreabilidade

Liga cada requisito à forma como ele foi comprovado. Os números se referem aos testes da seção 12.

COBERTO: comprovado por teste automatizado · **PARCIAL**: teste cobre só parte da regra · **LACUNA**: implementado, sem teste · **MANUAL**: conferido por uso direto · **REVISADO**: conferido por revisão de código ou modelo · **VERIFICADO**: conferido por inspeção.

Requisito	História	Como foi verificado	Situação
RF001	HU01	Teste 6 (lista pública e paginada)	COBERTO
RF002	HU02	Teste 7 (filtro por categoria)	COBERTO
RF003	HU03	Teste 9 (busca por texto)	COBERTO
RF004	HU04	Teste 10 (ordenação por preço)	COBERTO
RF005	HU05	Teste 8 (filtro por disponibilidade)	COBERTO
RF006	HU06	Implementado, **sem teste automatizado**	LACUNA
RF007	HU07	Teste 12 (ordem alfabética e contagem)	COBERTO
RF008	HU08	Verificação manual: acesso ao Django Admin no computador do responsável	MANUAL
RF009	—	Teste 5 (endpoint de saúde)	COBERTO
RN001	—	Teste 1 (preço zero rejeitado pelo banco)	COBERTO
RN002	—	Testes 2 e 3 (duplicidade e outra unidade)	COBERTO
RN003	HU08	Teste 4 cobre o produtor. Categoria e unidade usam a mesma regra (PROTECT), mas sem teste próprio	PARCIAL
RN004	—	Teste 11 (produtor inativo some do catálogo)	COBERTO
RN005	—	Regra no banco (UNIQUE), **sem teste automatizado**	LACUNA
RN006	—	Teste 14 (todos os produtores marcados como fictícios)	COBERTO
RNF001	—	Teste 13 (visitante não cria produto)	COBERTO
RNF002	—	Inspeção do histórico Git antes da publicação: nenhuma senha ou chave encontrada	VERIFICADO
RNF003	—	Ambiente recriado no computador do responsável. Falta a reprodução por outro integrante	PENDENTE
RNF004	—	`python manage.py test catalogo`: 14 testes aprovados	COBERTO
RNF005	—	Revisão de código: paginação e `select_related` na listagem	REVISADO
RNF006	—	Revisão do modelo de dados: não há campos de CPF, CNPJ ou endereço	REVISADO

> **Por que mostrar as lacunas?**
> Uma matriz que aponta o que ainda **não** está coberto é mais útil e mais honesta do que uma com tudo "verde". As lacunas de RF006 e RN005 viraram um item do Sprint Backlog.

Sprint Backlog (proposto)

#	Item	Relaciona-se a	Situação
1	Ambiente de desenvolvimento com Docker e PostgreSQL	RNF003	CONCLUÍDO
2	Backend Django + DRF conectado ao banco	Base técnica	CONCLUÍDO
3	Modelo de dados do catálogo com regras de integridade	RN001–RN005	CONCLUÍDO
4	API de consulta com filtros, busca e ordenação	HU01–HU07	CONCLUÍDO
5	Painel administrativo (Django Admin)	HU08	CONCLUÍDO
6	Dados fictícios de demonstração	RN006	CONCLUÍDO
7	Testes automatizados (14)	RNF004	CONCLUÍDO
8	Documentação técnica e repositório no GitHub	RNF002, RNF003	CONCLUÍDO
9	Testes para as lacunas (RF006, RN005, RN003 para categoria e unidade)	RNF004	PENDENTE
10	Reprodução do ambiente por outro integrante	RNF003	PENDENTE
11	Roteiro e ensaio da demonstração da Sprint Review	Sprint Goal	PENDENTE
12	Tela de catálogo no frontend (Next.js)	HU01–HU05	EM ANÁLISE

O item 12 depende de decisão da equipe e do tempo disponível. Sem ele, o Sprint Goal continua sendo demonstrável pela API navegável e pelo Django Admin.

Definition of Done (proposta)

Um item só é considerado **pronto** quando atende a todos os critérios abaixo:

- O código está no GitHub, no branch `main`, com mensagem de commit no padrão Conventional Commits.
- Todos os testes automatizados passam (`python manage.py test`).
- Mudanças no banco estão em migrations versionadas.
- Nenhuma senha, chave ou arquivo `.env` foi versionado.
- A documentação foi atualizada quando o item muda a forma de instalar, acessar ou usar o sistema.
- O item foi revisado e demonstrado ao responsável técnico.

Artefatos e compromissos do Scrum

Artefato	Compromisso	No Raiz Direta
Product Backlog	Product Goal	A definir pela equipe (ex.: conectar produtores locais a compradores profissionais)
Sprint Backlog	Sprint Goal	Tabela e objetivo propostos nesta seção
Increment	Definition of Done	Catálogo funcional e testado, avaliado pela DoD acima

Referência: Scrum Guide (novembro de 2020), de Ken Schwaber e Jeff Sutherland.

18 Situação atual e próximos passos

Pendências imediatas

#	Ação	Responsável	Situação
1	Convidar os integrantes como colaboradores no GitHub	Peres	PENDENTE
2	Cada integrante: seguir a seção 7 e confirmar que o projeto roda	Equipe	PENDENTE
3	Revisar e aprovar a proposta da seção 17 (Sprint Goal, histórias, Definition of Done)	Equipe	PENDENTE
4	Adicionar testes para os filtros por produtor e por município e para a regra RN005 (lacunas da matriz de rastreabilidade)	Peres	PENDENTE

Próximas etapas sugeridas

Etapa	Descrição	Situação
Frontend do catálogo	Telas em Next.js consumindo a API: listagem, filtros e página de produto.	PLANEJADO
Requisitos formais	Proposta na seção 17. Falta a aprovação da equipe e a validação com a comunidade.	EM ANÁLISE
Roteiro da demonstração	Sequência da apresentação: Admin → API com filtros → testes passando.	PLANEJADO
Autenticação e perfis	Login de produtor e comprador, com endpoints de escrita protegidos.	PLANEJADO
Pedidos	Carrinho, pedido e itens do pedido, com integridade transacional.	PLANEJADO

> **Validação com a comunidade (Extensão I)**
>
> As categorias, unidades e campos do modelo foram definidos como proposta inicial. Antes de tratá-los como definitivos, é importante validá-los com produtores e compradores reais, por exemplo em entrevistas ou questionários. Essa validação também gera as evidências exigidas pela disciplina de Extensão.

A Glossário

Termo	Significado
API REST	Forma padronizada de um sistema oferecer dados a outros sistemas por meio de endereços web (endpoints) que devolvem JSON.
Contêiner	Programa empacotado com tudo de que precisa, rodando isolado, igual em qualquer computador.
Docker Compose	Arquivo (`docker-compose.yml`) que descreve e liga vários contêineres com um único comando.
Volume	Espaço onde o Docker guarda dados que devem sobreviver quando o contêiner é removido.
Ambiente virtual (.venv)	Pasta com um Python e bibliotecas exclusivos do projeto, isolados do resto do computador.
ORM	Camada que permite manipular o banco com objetos Python em vez de escrever SQL à mão. O do Django faz isso.
Migration	Arquivo versionado que descreve uma mudança na estrutura do banco (criar tabela, adicionar campo).
Seed	Dados iniciais inseridos para testar o sistema. Aqui, o comando `popular_dados` .
Endpoint	Endereço específico da API, como `/api/produtos/` .
Serializer	Componente do DRF que converte objetos do banco em JSON e define o contrato da API.
PK / FK	Chave primária (identifica cada linha) e chave estrangeira (aponta para uma linha de outra tabela).
Constraint	Regra garantida pelo próprio banco, como CHECK, UNIQUE e NOT NULL.
Slug	Versão simplificada de um nome para uso em URLs (ex.: "Raízes e Tubérculos" → `raizes-e-tuberculos`).
Healthcheck	Verificação automática de que um serviço está pronto. No projeto, o banco aparece como *healthy*.
LTS	Long-Term Support: versão com suporte estendido de correções de segurança.
ADR	Architecture Decision Record: registro de uma decisão técnica com contexto e consequências.
Problema N+1	Lentidão causada por uma consulta extra ao banco para cada item de uma lista. É evitado com `select_related` .